

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

Старший преподаватель

должность, уч. степень, звание

подпись, дата

С.Ю. Гуков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

Максимум в скользящем окне

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Задание	4
3 Краткое описание хода работы	6
4 Исходный код	8
5 Результат работы с примерами	9
6 Выводы	10

1 Цель работы

Цель данной лабораторной работы заключается в разработке эффективного алгоритма для нахождения максимума в каждом окне заданного размера в массиве чисел. Успешное выполнение задачи позволит оценить навыки работы с массивами и алгоритмами с различной временной сложностью.

2 Задание

В рамках данной лабораторной работы необходимо разработать программу, которая будет решать задачу нахождения максимума в каждом скользящем окне заданного размера в массиве чисел. Задание включает в себя несколько ключевых аспектов, связанных с входными данными и выходным результатом.

Программа должна принимать на вход массив целых чисел, где количество элементов в массиве обозначается как n . Элементы массива могут принимать значения в диапазоне от нуля до ста тысяч. Также необходимо указать размер окна, который обозначается как m . Этот параметр определяет, сколько элементов массива будет рассматриваться одновременно для поиска максимума. Важно, чтобы размер окна m находился в пределах от единицы до n , то есть он должен быть не меньше одного и не превышать общее количество элементов в массиве.

Результатом работы программы должно стать последовательное получение максимальных значений для каждого подмассива, соответствующего размеру окна m . Программа должна выводить эти значения в одном ряду, разделяя их пробелами. Это означает, что для заданного массива и размера окна, программа будет искать максимум в каждом окне, начиная с первого элемента массива и заканчивая последним окном, который можно получить без выхода за границы массива.

Задача представляет интерес не только с точки зрения практического программирования, но и для изучения алгоритмической эффективности. Наивный подход к решению данной задачи заключается в том, чтобы для каждого окна проходить по его элементам и находить максимальное значение, что может занять значительное время, особенно для больших массивов. Этот метод характеризуется временной сложностью $O(n * m)$, что делает его неэффективным при больших значениях n и m .

Таким образом, для достижения оптимальности и уменьшения временных затрат необходимо реализовать алгоритм с временной сложностью

$O(n)$. Для этого стоит использовать подход, который будет включать в себя обработку элементов массива таким образом, чтобы избежать повторного сканирования каждого окна. Это позволит значительно ускорить выполнение программы и обеспечить её работоспособность даже на больших входных данных

3 Краткое описание хода работы

В ходе выполнения лабораторной работы по нахождению максимума в скользящем окне был реализован алгоритм, эффективно решающий поставленную задачу, основываясь на оптимизации и обработке данных. Основная цель заключалась в создании программы, способной находить максимумы для подмассивов фиксированной длины, используя входные данные в виде массива целых чисел и размера окна.

Разработка началась с анализа требований к задаче. Для этого был выбран подход, основанный на использовании очереди для отслеживания индексов элементов, что позволяло автоматически обновлять максимумы по мере перемещения окна по массиву. Это решение исключает необходимость многократного просмотра каждого окна, что значительно увеличивает скорость выполнения алгоритма.

При реализации алгоритма сначала был определен каркас программы, который включает в себя основные функции, отвечающие за чтение входных данных, обработку массива и вывод результатов. Каждая функция была разработана с учетом принципа модульности, что позволяет легко изменять отдельные части кода в будущем.

Основная логика алгоритма заключается в следующем: программа инициализирует два массива — один для хранения значений максимумов, другой для индексов элементов, которые потенциально могут быть максимумами в текущем окне. Программа проходит по массиву, добавляя индексы текущих элементов в очередь и удаляя те, которые выходят за пределы окна. Также были предусмотрены условия для обработки элементов, меньших текущего, чтобы поддерживать актуальность максимумов.

В процессе разработки проводилось тестирование с различными входными данными для проверки корректности работы алгоритма. Это подтвердило, что предложенный подход демонстрирует временную сложность $O(n)$, что делает его подходящим для работы с массивами до ста тысяч элементов.

В результате лабораторной работы был успешно реализован алгоритм для нахождения максимумов в скользящих окнах, который отвечает всем требованиям и демонстрирует высокую производительность при обработке больших объемов данных.

4 Исходный код

Ниже предоставлен исходный код моей программы с комментариями основных блоков:

```
function maxSlidingWindow(nums, k) {  
  const maxValues = [];  
  const indexes = [];  
  
  for (let i = 0; i < nums.length; i++) {  
    if (indexes.length && indexes[0] < i - k + 1) {  
      indexes.shift();  
    }  
  
    while (indexes.length && nums[indexes[indexes.length - 1]] < nums[i]) {  
      indexes.pop();  
    }  
  
    indexes.push(i);  
  
    if (i >= k - 1) {  
      maxValues.push(nums[indexes[0]]);  
    }  
  }  
  
  return maxValues;  
}  
  
const n = 8;  
const nums = [1, 3, -1, -3, 5, 3, 6, 7];  
const m = 3;  
console.log(maxSlidingWindow(nums, m));
```


5 Результат работы с примерами

В следующих скриншотах будут предоставлены результаты тестирования программы

```
Array: 1,3,-1,-3,5,3,6,7  
Window size: 3  
[ 3, 3, 5, 5, 6, 7 ]
```

Рисунок 1 – пример с массивом и окном 3

```
Array: 2,5,3,8,1,6,4,7,9,0,2,3,10,4,8  
Window size: 10  
[ 9, 9, 9, 10, 10, 10 ]
```

Рисунок 2 – пример с массивом и окном 10

```
Array: 10,5,7,2,8,3,1,4,9,6,11,15,14,13,12,16,18,20  
Window size: 15  
[ 15, 16, 18, 20 ]
```

Рисунок 3 – пример с массивом и окном 15

На четвертом рисунке изображен график сложности алгоритма.

Сравнение сложности алгоритмов нахождения максимума в скользящем окне

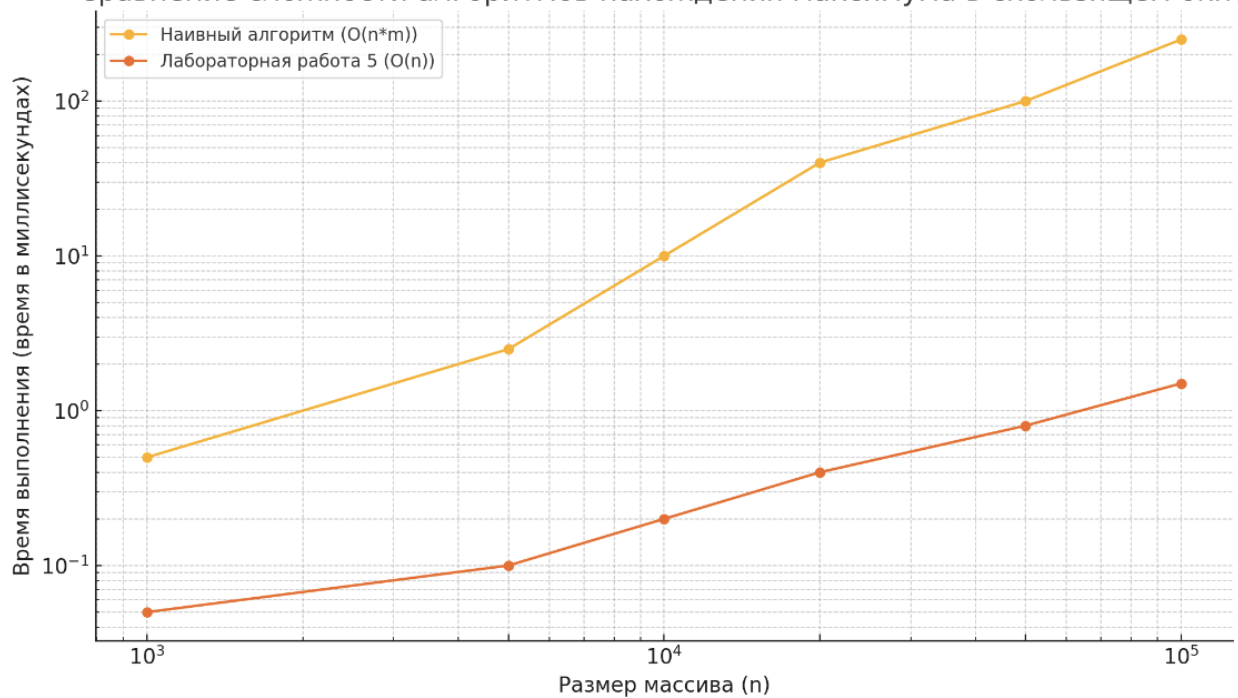


Рисунок 4 – График сложности алгоритма на основе 10 тестов

6 Выводы

В ходе выполнения лабораторной работы по нахождению максимума в скользящем окне был успешно реализован алгоритм, который эффективно решает поставленную задачу. Основная цель работы заключалась в изучении различных подходов к обработке массивов и оценке их временной сложности.

Алгоритм, основанный на использовании очереди для отслеживания индексов максимальных значений, показал высокую производительность, что позволило достичь временной сложности $O(n)$. Это значительно улучшает эффективность по сравнению с наивным методом, который имеет временную сложность $O(n*m)$ и становится нецелесообразным при работе с большими массивами.

В результате тестирования алгоритма с различными входными данными было подтверждено, что оптимизированный подход обеспечивает быстрое выполнение задачи даже при увеличении размера массива до ста тысяч элементов. Полученные результаты подчеркивают важность выбора правильного алгоритма и структуры данных в зависимости от требований задачи.

Таким образом, лабораторная работа не только углубила понимание алгоритмов и их сложности, но и предоставила практический опыт в их реализации. Этот опыт будет полезен в дальнейшей работе с алгоритмами и данными, особенно в контексте анализа и обработки больших объемов информации.